Name: Cohort/Batch: Date: Score:

JULIA PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of Julia? Threat Model

Q2 How does Julia handle type checking: static or dynamic? Architecture

Q3 How does Julia manage memory? Configuration

Q4 What is Julia's concurrency model? Hardening

Q5 How are packages and dependencies managed in Julia? SSO / IdP

Q6 How does Julia implement object-oriented or functional programming? Observability

Q7 How does Julia handle errors? Lifecycle

Q8 What testing tools are commonly used in Julia? Compliance

Q9 What makes Julia well-suited for its target domain? Testing

Q10 What are common performance pitfalls in Julia? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 **Julia was designed with specific priorities around** Mitigation

Julia was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A2 **Julia uses its type system to catch** Primitives

Julia uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A3 **Julia uses one of three approaches: manual** Hardening

Julia uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A4 **Julia supports concurrency through threads, coroutines,** Gotchas

Julia supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A5 **Julia uses a package manager and a** Federation

Julia uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A6 **Julia supports classes and inheritance for OOP,** Observability

Julia supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern Julia programs blend both paradigms depending on the problem.

A7 **Julia surfaces errors via exceptions, Result/Either types,** Automation

Julia surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A8 **Julia has a standard or widely adopted** Governance

Julia has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A9 **Julia excels in its domain due to** Assessment

Julia excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A10 **Typical performance issues in Julia include excessive** Optimization

Typical performance issues in Julia include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.